

STAT534 HW2

Contents

Setups	1
Packages	1
Data	1
Problem 1: Logistic Regression AIC	1
Part A: Solution	1
Part B: AI Solution	2
Part C: Comparison	4
Problem 2: Forward Greedy Search	4
Part A: Solution	4
Part B: AI Solution	6
Part C: Comparison	7
Problem 3: Backward Greedy Search	8
Part A: Solution	8
Part B: AI Solution	9
Part C: Comparison	11
Problem 4: Explorations	12
Part A: Solution	12
Part B: AI Solution	13
Part C: Comparison	16

Setups

Packages

```
library(data.table)
```

Data

```
df <- fread("hw2/534binarydata.txt")
```

Problem 1: Logistic Regression AIC

Part A: Solution

I design the AIC function to be inherited from a base function `getLogisticKIC`, which has the option to calculate the AIC or BIC as the criterion.

One thing is here I suppress the warning generate from AIC/BIC calculation. The issue is mostly caused by the collinearity, yet the guideline doesn't specifically mention how we should process encountering this issue.

Thus, I will just leave it with TODO flag, and suppress warnings for now.

```
getLogisticKIC <- function(response, explanatory, data, type = "A") {
  #'
  #' Args:
  #' -----
  #' - response (int): the response variable column index.
  #' - explanatory (vector[int]): the vector/set of explanatory variables column indexes.
  #' - data (data.frame like): the target data in data.frame-like object.
  #' - type (str): {A, B}, denoting kic, BIC, respectively. Default to use AIC.
  #'
  #' Return
  #' -----
  #' - KIC (num): KIC value.

  # arg check
  if (!type %in% c("A", "B")) {stop(sprintf("Type %s is not allowed. Should be 'A' or 'B'", type))}

  # accomodate different data inputs
  data <- as.matrix(data)

  # logistic regression model
  if(!length(explanatory)){
    # empty variable case
    model <- glm(data[, response] ~ 1, family = binomial(link='logit'))
  } else {
    # regular case
    model <- glm(data[,response] ~ data[, as.numeric(explanatory)],
                  family = binomial(link='logit'))
  }

  # KIC calc.
  multiplier <- ifelse(type == "A", 2, log(nrow(data)))
  deviance_ <- model$deviance
  kic <- deviance_ + multiplier * (1 + length(explanatory))

  return(kic)
}

getLogisticAIC <- function(response, explanatory, data) {
  #TODO: there's warnings about the glm estimation, currently suppress, worth investigating.
  return(getLogisticKIC(response, explanatory, data, type = "A") |> suppressWarnings())
}

getLogisticBIC <- function(response, explanatory, data) {
  #TODO: there's warnings about the glm estimation, currently suppress, worth investigating.
  return(getLogisticKIC(response, explanatory, data, type = "B") |> suppressWarnings())
}
```

Part B: AI Solution

Prompt

Please design a function named `getLogisticAIC` that uses `glm` to fit a logistic regression, and return the $AIC = deviance + 2 * \text{NumberOfRegressionCoefs}$. The function should come with proper doc-string, also

handles the edge cases.

Edge case: 1. empty model: when there's no explanatory vars provided, run constant regression. 2. also remember to handle different input data type cases

The function should be designed as follow:

an BIC function is provided as example:

Code Generated

```
## Compute the AIC for a Logistic Regression Model
##
## This function fits a logistic regression using `glm` and calculates the
## Akaike Information Criterion (AIC) based on the formula:
## AIC = deviance + 2 * NumberOfRegressionCoefficients.
##
## @param response (int): The index of the response variable in the data.
## @param explanatory (vector[int]): A vector of indices for the explanatory variables.
## @param data (df/dt/matrix): The dataset containing the variables.
##
## @return (numeric): The calculated AIC value.
AI_getLogisticAIC <- function(response, explanatory, data) {

  # Ensure data is in a format compatible with index-based subsetting
  # This handles data.table and matrix edge cases for glm
  if (!is.data.frame(data)) {
    data <- as.data.frame(data)
  }

  # Extract the response vector
  y <- data[, response]

  # Case 1: Empty model (Intercept only)
  if (0 == length(explanatory)) {
    fit <- glm(y ~ 1, family = binomial(link = "logit"))
    deviance <- fit$deviance
  }
  # Case 2: Model with explanatory variables
  else {
    # Extract explanatory variables as a matrix
    x <- as.matrix(data[, explanatory, drop = FALSE])
    fit <- glm(y ~ x, family = binomial(link = "logit"))
    deviance <- fit$deviance
  }

  # Calculate Number of Regression Coefficients
  # (Number of predictors + 1 for the intercept)
  k <- 1 + length(explanatory)

  # AIC Formula: Deviance + 2 * k
  aic_value <- deviance + 2 * k

  return(aic_value)
}
```

Part C: Comparison

Correctness

Both my and AI's solutions give the correct AIC values. I check with built-in AIC function of R.

```
getLogisticAIC(ncol(df), c(1, 3, 5), df)
```

```
## [1] 147.6771
```

```
AI_getLogisticAIC(ncol(as.data.frame(df)), c(1, 3, 5), as.data.frame(df))
```

```
## [1] 147.6771
```

```
m <- as.matrix(df)
```

```
AIC(glm(m[, ncol(df)] ~ m[, c(1, 3, 5)], family = binomial(link = 'logit')))
```

```
## [1] 147.6771
```

Readability

Regarding the readability, I think we are the same generally. We adopted different doc string styles. Ai solution is more specific about the function desc, yet we both provide sufficient function usages.

Another thing is AI's solution uses `k <- 1 + length(explanatory)` in the AIC calculation, which I think will be subtly clearer when you read the function w/o background of AIC/BIC.

Algo & DS

- Function Design
 - I think my design is more general and scalable to accomodate future extension to different criteria e.g. BIC
- Data Structure
 - we both choose matrix for the calculation, so I think this part won't be too much different

Robustness

AI's solution is not accepting data.table object, which is rather common in R. My solution handles this by converting the data input first, then proceed to main logic.

Problem 2: Forward Greedy Search

Part A: Solution

I design a base `forwardSearch` algorithm function to accomodate more criteria functions.

```
forwardSearch <- function(response, data, KICfunc) {  
  #'  
  #' Args  
  #' ----  
  #' - response (int): response var index.  
  #' - data (df | dt | matrix): target data in data.frame like object.  
  #' - KICfunc (function): function to compute AIC/BIC value.  
  #'  
  #' Returns  
  #' -----  
  #' - A list with:  
  #'   - model: selected explanatory variable indices.  
  #'   - kic: criterion value of the final model.  
}
```

```

#' - path: order of variables added during forward search.

# initial setups
all_vars <- setdiff(1:ncol(data), response)
n_all_vars <- length(all_vars)

explanatory <- c()

# initial kic
kic_curr <- KICfunc(
  response = response,
  explanatory = explanatory,
  data = data
)

while((length(explanatory) < n_all_vars)) {

  # add new variable from remaining set
  available_vars_idx <- setdiff(all_vars, explanatory)
  kics <- sapply(available_vars_idx, function(e) {KICfunc(response, union(explanatory, e), data)})

  if (kic_curr < min(kics)) {
    # case: stop when there no kic improvement
    break
  }

  # regular case: add variable to model that has min kic
  explanatory <- c(explanatory, available_vars_idx[which.min(kics)])
  kic_curr <- min(kics)
}

return(list(model=sort(explanatory), kic=kic_curr, path=explanatory))
}

forwardSearchAIC <- function(response, data) {
  return(forwardSearch(response, data, getLogisticAIC))
}

```

The data empirical results:

```

aic_forward_res <- forwardSearchAIC(response = ncol(df), data = df)
aic_forward_res

```

```

## $model
## [1] 3 4 20 21 22 23 34 46 49 53
##
## $kic
## [1] 22
##
## $path
## [1] 23 3 22 21 49 4 20 34 53 46

```

Part B: AI Solution

Prompt

Design the forward search algorithm function which finds the model with smallest AIC value.

The algorithm starts with empty model (constant model), then in r iteration: 1. If all vars are in the model, then stop, since there's no available var to add 2. Consider all the logistic regressions that are obtained by adding one variable, use the `getLogisticAIC` function to compute AIC. 3. Find the logistic regression model with the smallest AIC 4. if the AIC for adding this best var is smaller, add it to current model, then proceed to next iteration.

Also, remember to add doc-string and try to make function robust with edge cases considered. The design pattern should be like:

Code Generated

```
## Forward Stepwise Selection using AIC
##
## This function performs a forward search to find the optimal logistic regression
## model. It starts with an empty model and adds variables one by one based on
## which addition yields the lowest AIC, provided it is lower than the current AIC.
##
## @param response (int): The index of the response variable in the data.
## @param data (df|dt|matrix): The dataset containing the variables.
##
## @return (list): A list containing 'best_model' (indices of selected variables)
## and 'aic' (the AIC value of that model).
AI_forwardSearchAIC <- function(response, data) {

  # 1. Initialization and Data Handling
  if (!is.data.frame(data)) {
    data <- as.data.frame(data)
  }

  num_cols <- ncol(data)
  all_vars <- setdiff(1:num_cols, response)
  current_model <- c() # Start with empty model (intercept only)

  # Calculate initial AIC for the constant model
  current_aic <- AI_getLogisticAIC(response, current_model, data)

  # 2. Iterative Search
  repeat {
    # Check if all variables are already in the model
    remaining_vars <- setdiff(all_vars, current_model)
    if (length(remaining_vars) == 0) {
      break
    }

    # Track the best candidate in this iteration
    best_candidate_aic <- Inf
    best_candidate_var <- NULL

    # Test adding each available variable one by one
    for (var in remaining_vars) {
```

```

trial_vars <- c(current_model, var)
trial_aic <- AI_getLogisticAIC(response, trial_vars, data)

# If this is the best variable tested in THIS iteration
if (trial_aic < best_candidate_aic) {
  best_candidate_aic <- trial_aic
  best_candidate_var <- var
}
}

# 3. Improvement Check
# If the best candidate improves (lowers) the AIC, add it
if (best_candidate_aic < current_aic) {
  current_model <- c(current_model, best_candidate_var)
  current_aic <- best_candidate_aic
} else {
  # No further improvement possible
  break
}
}

# 4. Output results
return(list(
  best_model = current_model,
  aic = current_aic
))
}

AI_aic_forward_res <- AI_forwardSearchAIC(ncol(df), as.data.frame(df)) |> suppressWarnings()

```

Part C: Comparison

Correctness

We both have the same AIC value and the best model given the data.

```
aic_forward_res
```

```
## $model
## [1] 3 4 20 21 22 23 34 46 49 53
##
## $kic
## [1] 22
##
## $path
## [1] 23 3 22 21 49 4 20 34 53 46

```

```
AI_aic_forward_res
```

```
## $best_model
## [1] 23 3 22 21 49 4 20 34 53 46
##
## $aic
## [1] 22

```

Readability

I think the AI's solution will be more friendly to ppl who has no knowledge to R and the forward/backward selection, since it writes comments and notes for every steps. The overall readability is better in my opinion.

Yet I think my solution is sufficiently readable for users who has prior knowledge to R's apply functions, and the main steps for the searching algo are documented as well.

Regarding the iteration, I think the repeat loop used by AI's solution will also be more readable since it does not require user to initial variables outside the loop.

Algo & DS

The data structures used by both solution do not differ that much. We both use simple vector and apply set operations to get the explanatory vars. One performance difference here is my solution use `sapply` function to vectorize the AIC calculation for new models, while AI uses loop. Vectorized function would be faster in this case, while another will be more readable.

Robustness

We both take care of the response-not-in-the-last-column case. However, AI did not accept `data.table` for the data input type; my solution accommodate this common data type.

P.S. Another aspect is that I think repeat syntax is easier to implement w/o errors than while, due to its simplicity in this case. (it's not code robustness, but more implementation robustness).

Problem 3: Backward Greedy Search

Part A: Solution

Follow similar idea, I write a base backward search, and extend it to use AIC.

```
backwardSearch <- function(response, data, KICfunc) {  
  #'  
  #' Args  
  #' ----  
  #' - response (int): response var index.  
  #' - data (df | dt | matrix): target data in data.frame like object.  
  #' - KICfunc (function): function to compute AIC/BIC value.  
  #'  
  #' Returns  
  #' -----  
  #' - A list with:  
  #'   - model: selected explanatory variable indices.  
  #'   - kic: criterion value of the final model.  
  #'   - path: order of variables added during backward search.  
  
  # initial setups  
  all_vars <- setdiff(1:ncol(data), response)  
  n_all_vars <- length(all_vars)  
  explanatory <- all_vars  
  
  # initial kic  
  kic_curr <- KICfunc(  
    response = response,  
    explanatory = explanatory,  
    data = data
```



```

)

while((length(explanatory) > 0)) {

  # remove variable from the explanatory set
  available_vars_idx <- explanatory
  kics <- sapply(available_vars_idx, function(e) {KICfunc(response, setdiff(explanatory, e), data)})

  if (kic_curr < min(kics)) {
    # case: stop when there no kic improvement
    break
  }

  # regular case: remove variable from model that has min kic
  explanatory <- setdiff(explanatory, available_vars_idx[which.min(kics)])
  kic_curr <- min(kics)
}

return(list(model=sort(explanatory), kic=kic_curr, path=explanatory))
}

backwardSearchAIC <- function(response, data) {
  return(backwardSearch(response, data, getLogisticAIC))
}

```

The empirical results:

```

aic_backward_res <- backwardSearchAIC(response = ncol(df), data = df)
aic_backward_res

## $model
## [1] 1 3 9 12 20 23 25 34 41 46
##
## $kic
## [1] 22
##
## $path
## [1] 1 3 9 12 20 23 25 34 41 46

```

Part B: AI Solution

Prompt

Similarly, design a function that performs backward search. First start with the full model, then in each iteration: 1. if the current model is the empty model (i.e., constant), stop 2. consider all logistic models by deleting one variable from existing model, calc. the AIC. 3. find the model that has the smallest AIC 4. if the smallest AIC by removing one variable is smaller than existing one, then remove it from existing model and proceed to next iteration. The function should be designed as follow:

Code Generated

```

#' Backward Stepwise Selection using AIC
#'
#' This function performs a backward search to find the optimal logistic regression
#' model. It starts with the full model (all available variables) and removes

```

```

#' variables one by one if the removal results in a lower AIC.
#'
#' @param response (int): The index of the response variable in the data.
#' @param data (df|dt|matrix): The dataset containing the variables.
#'
#' @return (list): A list containing 'best_model' (indices of selected variables)
#' and 'aic' (the AIC value of that model).
AI_backwardSearchAIC <- function(response, data) {

  # 1. Initialization
  if (!is.data.frame(data)) {
    data <- as.data.frame(data)
  }

  num_cols <- ncol(data)
  # Start with all variables except the response
  current_model <- setdiff(1:num_cols, response)

  # Calculate initial AIC for the full model
  current_aic <- AI_getLogisticAIC(response, current_model, data)

  # 2. Iterative Search
  repeat {
    # Edge Case: If the model is already empty (intercept only), stop
    if (length(current_model) == 0) {
      break
    }

    # Track the best candidate for removal in this iteration
    best_candidate_aic <- Inf
    best_candidate_var <- NULL

    # Test removing each variable currently in the model one by one
    for (var in current_model) {
      # Create a trial model by excluding the current variable
      trial_vars <- setdiff(current_model, var)
      trial_aic <- AI_getLogisticAIC(response, trial_vars, data)

      # Find the removal that results in the smallest AIC
      if (trial_aic < best_candidate_aic) {
        best_candidate_aic <- trial_aic
        best_candidate_var <- var
      }
    }

    # 3. Improvement Check
    # If removing the "best" candidate results in a lower AIC than the current model
    if (best_candidate_aic < current_aic) {
      current_model <- setdiff(current_model, best_candidate_var)
      current_aic <- best_candidate_aic
    } else {
      # Removing any more variables increases the AIC; stop here.
      break
    }
  }
}

```

```

    }
  }

  # 4. Output results
  return(list(
    best_model = current_model,
    aic = current_aic
  ))
}

AI_aic_backward_res <- AI_backwardSearchAIC(ncol(df), as.data.frame(df)) |> suppressWarnings()

```

Part C: Comparison

Correctness

Both solutions yield the same AIC and model variable set.

```

aic_backward_res

## $model
## [1] 1 3 9 12 20 23 25 34 41 46
##
## $kic
## [1] 22
##
## $path
## [1] 1 3 9 12 20 23 25 34 41 46

AI_aic_backward_res

## $best_model
## [1] 1 3 9 12 20 23 25 34 41 46
##
## $aic
## [1] 22

```

Readability

Like it be mentioned above: 1. repeat is more readable than while loop 2. AI's comment is verbose but more informative to general users; My will require knowledge to R's apply syntax

Algo & DS

Also mentioned above: 1. both solutions use same data structure, vector, to store the results 2. sapply should be better than bland for loop in my opinion for performance reason 3. My function design is easier to extend to different criteria w/o repetitions, i.e., more scalable

Robustness

Same as Problem 2 - Part C: 1. data.table type is not properly handled by AI's solution, yet it is a common and powerful modern tool in R. 2. repeat syntax is easier for development

Problem 4: Explorations

Part A: Solution

First, I create the BIC versions for forward/backward search algos.

```
# forward search w/ BIC
forwardSearchBIC <- function(response, data) {
  return(forwardSearch(response, data, getLogisticBIC))
}

# backward search w/ BIC
backwardSearchBIC <- function(response, data) {
  return(backwardSearch(response, data, getLogisticBIC))
}
```

The empirical results:

```
# aic_forward_res <- forwardSearchAIC(response = ncol(df), data = df)
# aic_backward_res <- backwardSearchAIC(response = ncol(df), data = df)

bic_forward_res <- forwardSearchBIC(response = ncol(df), data = df)
bic_backward_res <- backwardSearchBIC(response = ncol(df), data = df)
```

Do forward and backward search w/ AIC yield the same results?

AIC forward and backward have the same AIC value, but they are from different variables set.

Here I think one possible explanation is that there exists collinearity, i.e., some variables are the combination of set of other vars. This will make different set of vars have the same AIC, since they are essentially the same. (e.g. $A = B + C$, $M(A) = M(B+C)$).

```
# output
print(aic_forward_res)

## $model
## [1] 3 4 20 21 22 23 34 46 49 53
##
## $kic
## [1] 22
##
## $path
## [1] 23 3 22 21 49 4 20 34 53 46

# model
cat("Difference between models: \n")

## Difference between models:
cat(setdiff(aic_forward_res$model, aic_backward_res$model), "\n\n")

## 4 21 22 49 53

# AIC
cat("Is the AIC/BIC the same?\n")

## Is the AIC/BIC the same?
cat(all.equal(aic_forward_res$kic, aic_backward_res$kic), "\n")

## TRUE
```

Do forward and backward search w/ BIC yield the same results?

Similar results are obtained in BIC's backward and forward case. BIC forward and backward have the same BIC value, but they are from different variables set, which is possibly caused by collinearity issue in the data.

```
# output
print(bic_forward_res)

## $model
## [1] 3 4 20 21 22 23 34 46 49 53
##
## $kic
## [1] 54.96934
##
## $path
## [1] 23 3 22 21 49 4 20 34 53 46

# model
cat("Difference between models: \n")

## Difference between models:
cat(setdiff(bic_forward_res$model, bic_backward_res$model), "\n\n")

## 4 21 22 49 53

# AIC
cat("Is the AIC/BIC the same?\n")

## Is the AIC/BIC the same?
cat(all.equal(bic_forward_res$kic, bic_backward_res$kic), "\n")

## TRUE
```

Do search w/ AIC and BIC yield the same results?

Both AIC and BIC will have the same best variables using forward/backward search, respectively, in this case. But when I use smaller dataset, I found the BIC will tend to choose less variable, which is reasonable by its formula.

```
print(aic_forward_res$model)

## [1] 3 4 20 21 22 23 34 46 49 53
print(bic_forward_res$model)

## [1] 3 4 20 21 22 23 34 46 49 53
print(aic_backward_res$model)

## [1] 1 3 9 12 20 23 25 34 41 46
print(bic_backward_res$model)

## [1] 1 3 9 12 20 23 25 34 41 46
```

Part B: AI Solution

Prompt

Repeat the following forward/backward search algos with BIC, create: `AI_forwardSearchBIC` and `AI_backwardSearchBIC` functions. Also remember to create intermediate `AI_getLogisticBIC` function

follow the above design patterns.

Code Generated

```
## Compute the BIC for a Logistic Regression Model
##
## Fits a logistic regression using `glm` and calculates the
## Bayesian Information Criterion (BIC) based on the formula:
## BIC = deviance + log(SampleSize) * NumberOfRegressionCoefficients.
##
## @param response (int): The index of the response variable.
## @param explanatory (vector[int]): Indices for the explanatory variables.
## @param data (df|dt|matrix): The dataset.
##
## @return (numeric): The calculated BIC value.
AI_getLogisticBIC <- function(response, explanatory, data) {

  # Handle data structure
  if (!is.data.frame(data)) {
    data <- as.data.frame(data)
  }

  n <- nrow(data)
  y <- data[, response]

  # Fit model based on presence of explanatory variables
  if (length(explanatory) == 0) {
    fit <- glm(y ~ 1, family = binomial(link = "logit"))
  } else {
    x <- as.matrix(data[, explanatory, drop = FALSE])
    fit <- glm(y ~ x, family = binomial(link = "logit"))
  }

  # BIC Formula: deviance + log(n) * (number of coefs)
  # k = length(explanatory) + 1 (for the intercept)
  k <- length(explanatory) + 1
  bic_value <- fit$deviance + log(n) * k

  return(bic_value)
}

## Forward Stepwise Selection using BIC
##
## Starts with an empty model and adds variables one by one
## if they yield the lowest BIC improvement.
##
## @param response (int): The index of the response variable.
## @param data (df|dt|matrix): The dataset.
##
## @return (list): Best model indices and its BIC value.
AI_forwardSearchBIC <- function(response, data) {
  if (!is.data.frame(data)) data <- as.data.frame(data)
```

```

all_vars <- setdiff(1:ncol(data), response)
current_model <- c()
current_bic <- AI_getLogisticBIC(response, current_model, data)

repeat {
  remaining_vars <- setdiff(all_vars, current_model)
  if (length(remaining_vars) == 0) break

  best_candidate_bic <- Inf
  best_candidate_var <- NULL

  for (var in remaining_vars) {
    trial_vars <- c(current_model, var)
    trial_bic <- AI_getLogisticBIC(response, trial_vars, data)

    if (trial_bic < best_candidate_bic) {
      best_candidate_bic <- trial_bic
      best_candidate_var <- var
    }
  }

  if (best_candidate_bic < current_bic) {
    current_model <- c(current_model, best_candidate_var)
    current_bic <- best_candidate_bic
  } else {
    break
  }
}
return(list(best_model = current_model, bic = current_bic))
}

## Backward Stepwise Selection using BIC
##
## Starts with the full model and removes variables one by one
## if the removal yields a lower BIC.
##
## @param response (int): The index of the response variable.
## @param data (df|dt|matrix): The dataset.
##
## @return (list): Best model indices and its BIC value.
AI_backwardSearchBIC <- function(response, data) {
  if (!is.data.frame(data)) data <- as.data.frame(data)

  current_model <- setdiff(1:ncol(data), response)
  current_bic <- AI_getLogisticBIC(response, current_model, data)

  repeat {
    if (length(current_model) == 0) break

    best_candidate_bic <- Inf
    best_candidate_var <- NULL

    for (var in current_model) {

```

```

trial_vars <- setdiff(current_model, var)
trial_bic <- AI_getLogisticBIC(response, trial_vars, data)

if (trial_bic < best_candidate_bic) {
  best_candidate_bic <- trial_bic
  best_candidate_var <- var
}

}

if (best_candidate_bic < current_bic) {
  current_model <- setdiff(current_model, best_candidate_var)
  current_bic <- best_candidate_bic
} else {
  break
}

}

return(list(best_model = current_model, bic = current_bic))
}

AI_bic_forward_res <- AI_forwardSearchBIC(ncol(df), as.data.frame(df)) |> suppressWarnings()
AI_bic_backward_res <- AI_backwardSearchBIC(ncol(df), as.data.frame(df)) |> suppressWarnings()

```

Part C: Comparison

Correctness

AI's BIC forward and backward have the same BIC values and model with my BIC implementations.

```
bic_forward_res
```

```

## $model
## [1] 3 4 20 21 22 23 34 46 49 53
##
## $kic
## [1] 54.96934
##
## $path
## [1] 23 3 22 21 49 4 20 34 53 46

```

```
AI_bic_forward_res
```

```

## $best_model
## [1] 23 3 22 21 49 4 20 34 53 46
##
## $bic
## [1] 54.96934

```

```
bic_backward_res
```

```

## $model
## [1] 1 3 9 12 20 23 25 34 41 46
##
## $kic
## [1] 54.96934
##
## $path
## [1] 1 3 9 12 20 23 25 34 41 46

```



```
AI_bic_backward_res
```

```
## $best_model
## [1] 1 3 9 12 20 23 25 34 41 46
##
## $bic
## [1] 54.96934
```

Readability

All like above, b/c it just change the AIC to BIC: 1. repeat syntax more readable 2. AI has more comments which in some case is redundant but generally informative

Algo & DS

Also mentioned above: 1. both solutions use same data structure, vector, to store the results 2. apply should be better than lapply for loop in my opinion for performance reason 3. My function design is easier to extend to different criteria w/o repetitions, i.e., more scalable

Robustness

Same as Problem 3 - Part C: 1. data.table type is not properly handled by AI's solution, yet it is a common and powerful modern tool in R. 2. repeat syntax is easier for development